🧾 FICHE MÉMO : Les fonctions en Python

🎯 Objectif

Les fonctions permettent de **regrouper un ensemble d'instructions** sous un même nom, afin de **réutiliser du code** facilement, de **le rendre plus clair** et de **limiter les répétitions**.

1️⃣ Définir une fonction

On utilise le mot-clé `def` pour **déclarer** une fonction.

```
def nom_de_fonction():
    # bloc d'instructions
```

Exemple simple :

```
def bonjour():
    print("Bonjour tout le monde !")

bonjour()  # appel de la fonction
```

2️⃣ Les paramètres

Les fonctions peuvent **recevoir des valeurs** (appelées *paramètres*).

```
def bonjour_prenom(prenom):
    print(f"Bonjour {prenom} !")

bonjour_prenom("Lina")
```

👉 Ici, `prenom` est un paramètre.

Quand on appelle la fonction, on lui passe une *valeur*.

3️⃣ Le mot-clé `return`

`return` permet à une fonction de **renvoyer une valeur**.

```
def addition(a, b):
    return a + b

resultat = addition(5, 3)
print(resultat)  # affiche 8
```

🧠 Une fonction peut **retourner** :

- un nombre
- une chaîne
- une liste
- ou même plusieurs valeurs à la fois.

```
def infos():
    return "Python", 3.12

langage, version = infos()
print(langage, version)
```

4️⃣ Paramètres par défaut

On peut définir des **valeurs par défaut** pour certains paramètres.

```
def bonjour(nom="inconnu"):
    print(f"Bonjour {nom} !")

bonjour()       # Bonjour inconnu !
bonjour("Lina")  # Bonjour Lina !
```

5️⃣ Arguments positionnels et nommés

Lors de l'appel, on peut passer les **valeurs dans l'ordre** (arguments positionnels)

ou **indiquer le nom du paramètre** (arguments nommés).

```
def infos(nom, age):
    print(f"{nom} a {age} ans")

# Appel avec arguments nommés :
infos(age=20, nom="Lina")

# Appel avec arguments positionnels :
infos("Lina", 20)
```

👉 Les deux fonctionnent !

Mais quand on utilise des **arguments nommés**, l'ordre **n'a plus d'importance**.

Alors qu'avec les **arguments positionnels**, **l'ordre doit être respecté**.

6️⃣ Fonctions avec conditions ou boucles

Les fonctions peuvent contenir **des conditions** et **des boucles** :

```
def compter(n):
    for i in range(1, n+1):
        print(i)

compter(5)
```

7️⃣ Bonnes pratiques

✅ Donner un **nom clair et descriptif** à la fonction

✅ Éviter qu'une fonction fasse trop de choses

✅ Toujours documenter avec un petit **commentaire** ou une **docstring**

```
def addition(a, b):
    """Retourne la somme de deux nombres."""
    return a + b
```

8️⃣ Récapitulatif rapide

Élément	Description
`def`	Définit une fonction
`return`	Retourne une valeur
Paramètre	Donnée envoyée à la fonction
Argument	Valeur passée lors de l'appel
Fonction anonyme	Peut être définie avec `lambda` (avancé)